ISTANBUL TECHNICAL UNIVERSITY

COMPUTER ENGINEERING DEPARTMENT

BLG 242E

DIGITAL CIRCUITS LABORATORY EXPERIMENT REPORT

EXPERIMENT NO	:	2
EXPERIMENT DATE	:	02.03.2026
LAB SESSION	:	MONDAY - 13.30
GROUP NO	:	G3

GROUP MEMBERS:

150230089	:	HÜSEYİN TEKE
150230033	:	KUTAY YEKTA GENÇOĞLU
15021044	:	HASAN BARIŞ ÇALIŞKAN

SPRING 2026

Contents

1 INTRODUCTION **1**

2 MATERIALS AND METHODS **1**

2.1 8:1 Multiplexer 1

2.2 BCD to 7-Segment Decoder 1

2.3 3:8 Decoder 4

2.4 Elaborated Design of all modules 5

3 RESULTS **6**

4 DISCUSSION **9**

5 CONCLUSION **10**

REFERENCES **11**

1 INTRODUCTION

The objective of this experiment is to design and implement complex combinational logic circuits using Verilog HDL on the Basys 3 FPGA board. The design process focuses on using combinational `always@(*)` blocks to realize various digital modules such as multiplexers, decoders, and encoders. Key concepts include the application of Verilog operators, understanding the hardware configuration of common anode seven-segment displays, and managing FPGA pin constraints.

2 MATERIALS AND METHODS

All designs were implemented in Verilog and targeted for the Artix-7 FPGAs.

2.1 8:1 Multiplexer

An 8:1 multiplexer was designed to route one of eight input lines (`in[7:0]`) to a single output `y` based on 3-bit select lines (`sel[2:0]`). As per the experiment requirements, the circuit was implemented using nested `if-else` structures within an `always@(*)` block instead of `switch-case` statements.

```
always @(*) begin
    if      (FunSel == 3'b000) LED = sw[0];
    else if (FunSel == 3'b001) LED = sw[1];
    else if (FunSel == 3'b010) LED = sw[2];
    else if (FunSel == 3'b011) LED = sw[3];
    else if (FunSel == 3'b100) LED = sw[4];
    else if (FunSel == 3'b101) LED = sw[5];
    else if (FunSel == 3'b110) LED = sw[6];
    else                       LED = sw[7];
end
```

Listing 1: 8:1 Mux Verilog Implementation

2.2 BCD to 7-Segment Decoder

A 4-bit BCD-to-7-segment decoder was developed to drive the segments `a-g` of the Basys 3 display. The mapping for digits 0-9 was defined using `case` statements. The design specifically accounts for the common anode configuration, where segments are active-low. For invalid BCD values (greater than 9), all segments were driven to a specific invalid pattern.

```
module BCDto7Segment(
    input wire [3:0] sw,
    output reg [6:0] seg,
    output reg dp,
    output reg an
    );
    /**
      an = anode
      reg[0] = A
      reg[1] = B
      reg[2] = C
      reg[3] = D
      reg[4] = E
      reg[5] = F
      reg[6] = G
      dp = DP
    **/
    always@(*)
    begin
        case(sw)
        4'b0000 :
        begin an = 1; dp = 1; seg = 7'b1000000;
        end
        4'b0001 :
        begin an = 1; dp = 1; seg = 7'b1111001;
        end
        4'b0010 :
        begin an = 1; dp = 1; seg = 7'b0100100;
        end
        4'b0011 :
        begin an = 1; dp = 1; seg = 7'b0110000;
        end
        4'b0100 :
        begin an = 1; dp = 1; seg = 7'b0011001;
        end
        4'b0101 :
        begin an = 1;  dp = 1; seg = 7'b0110000;
        end
        4'b0110 :
```

```
        begin an = 1; dp = 1; seg = 7'b000010;
        end
        4'b0111 :
        begin an = 1; dp = 1; seg = 7'b1111000;
        end
        4'b1000 :
        begin an = 1; dp = 1; seg = 7'b0000000;
        end
        4'b1001 :
        begin an = 1; dp = 1; seg = 7'b0010000;
        end
        default :
        begin an = 1; dp = 1; seg = 7'b1111111;
        end
        endcase
    end
endmodule
```

Listing 2: BCD to 7 Segment Display implementation

2.3 3:8 Decoder

A 3:8 decoder was implemented with a 3-bit input (`in[2:0]`) and 8-bit output (`out[7:0]`). The logic ensures that only one output line is active (high) for a given input value. The module was constructed using a single `always@(*)` block with `if-else` conditions to match each possible input value from 0 to 7.

```
module decoder_3to8(
    input [2:0] in,
    output reg [7:0] out
);
    always @(*) begin
        if (in == 3'b000) out = 8'b00000001;
        else if (in == 3'b001) out = 8'b00000010;
        else if (in == 3'b010) out = 8'b00000100;
        else if (in == 3'b011) out = 8'b00001000;
        else if (in == 3'b100) out = 8'b00010000;
        else if (in == 3'b101) out = 8'b00100000;
        else if (in == 3'b110) out = 8'b01000000;
        else if (in == 3'b111) out = 8'b10000000;
        else out = 8'b00000000;
    end
endmodule
```

Listing 3: 3:8 Decoder Verilog Implementation

2.4 Elaborated Design of all modules

The schematic generated by the Vivado Synthesis tool provides a visual representation of the logic gates and routing implemented by the Verilog code.

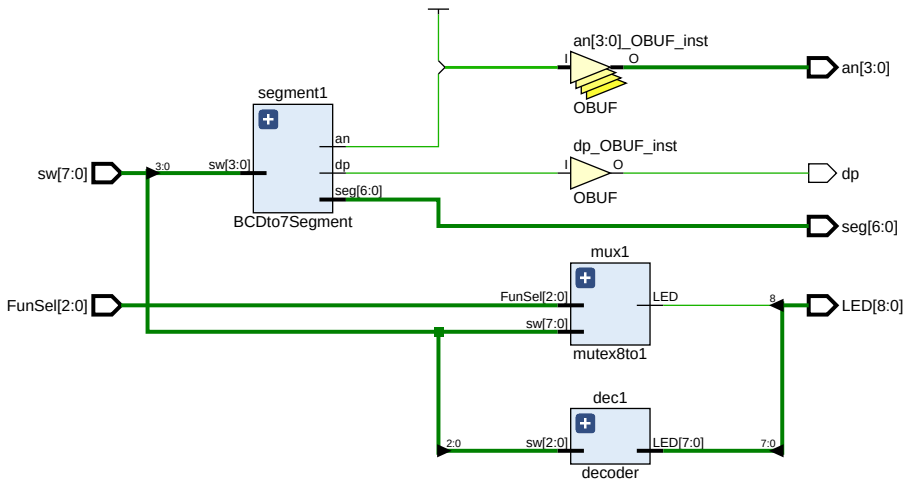

Figure 1: Elaborated Design schematic of the all modules.

3 RESULTS

The designs were verified through functional simulations and hardware testing.

- **Simulation:** Waveforms were generated to confirm correct output for all valid input ranges.

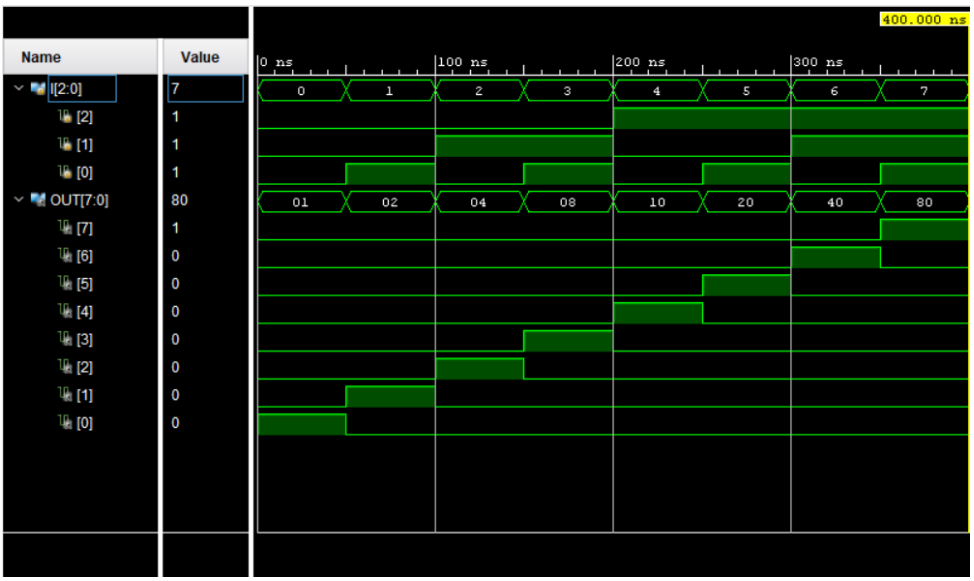

Figure 2: Decoder Simulation Results

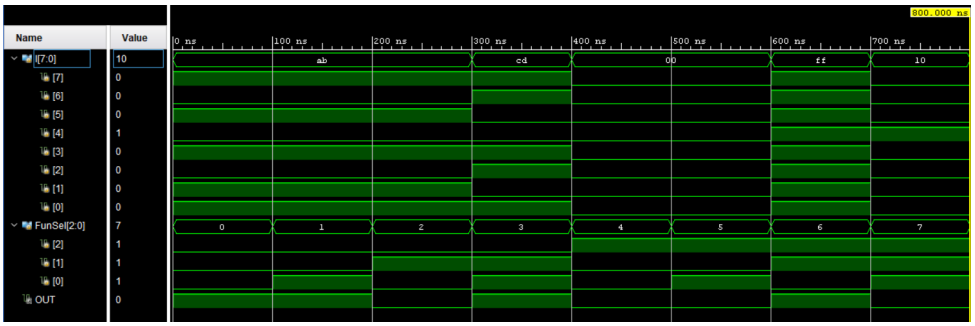

Figure 3: 8:1 Multiplexer Simulation Results

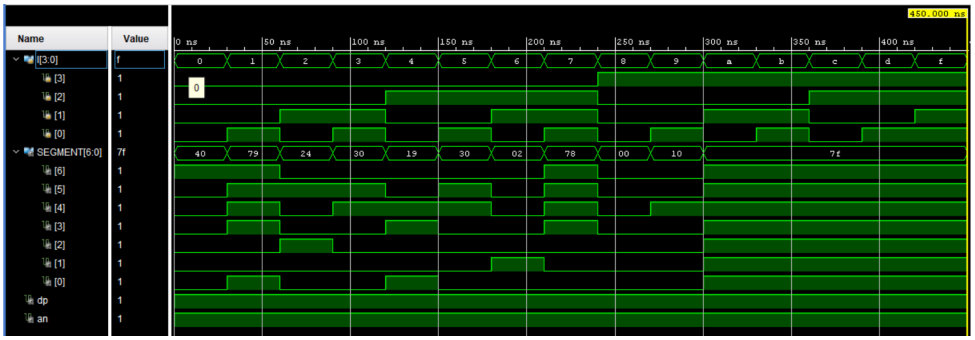

Figure 4: BCD Segment Simulation Results

- **Synthesis:** The synthesis reports indicate the resource usage, including the number of Look-Up Tables (LUTs) and carry logic utilized on the FPGA.
- **Hardware:** The inputs were mapped to DIP switches and outputs to the 7-segment display and LEDs to confirm real-time functionality.

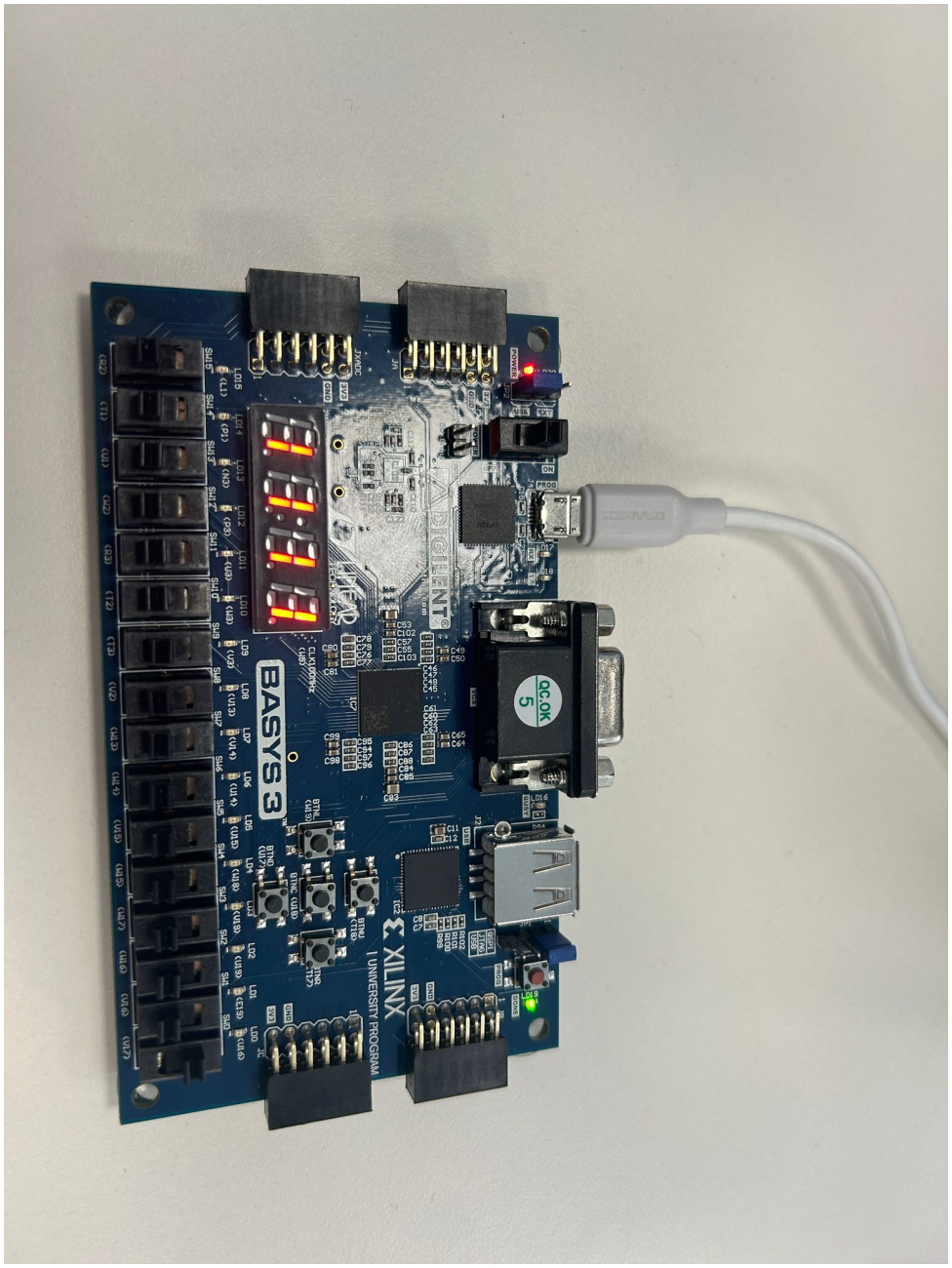

Figure 5: BCD Segment hardware result (one)

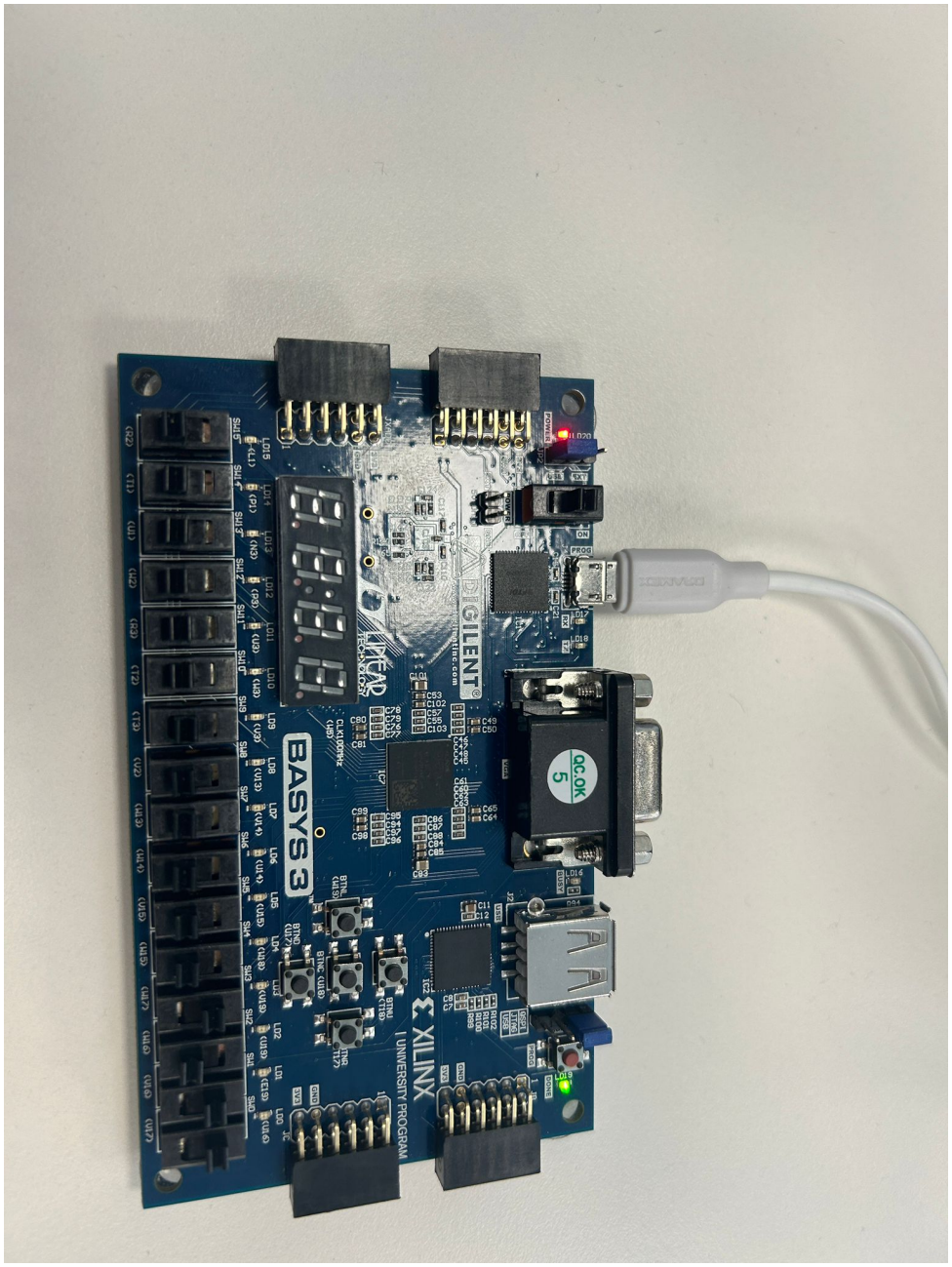

Figure 6: BCD Segment hardware result (undefined input)

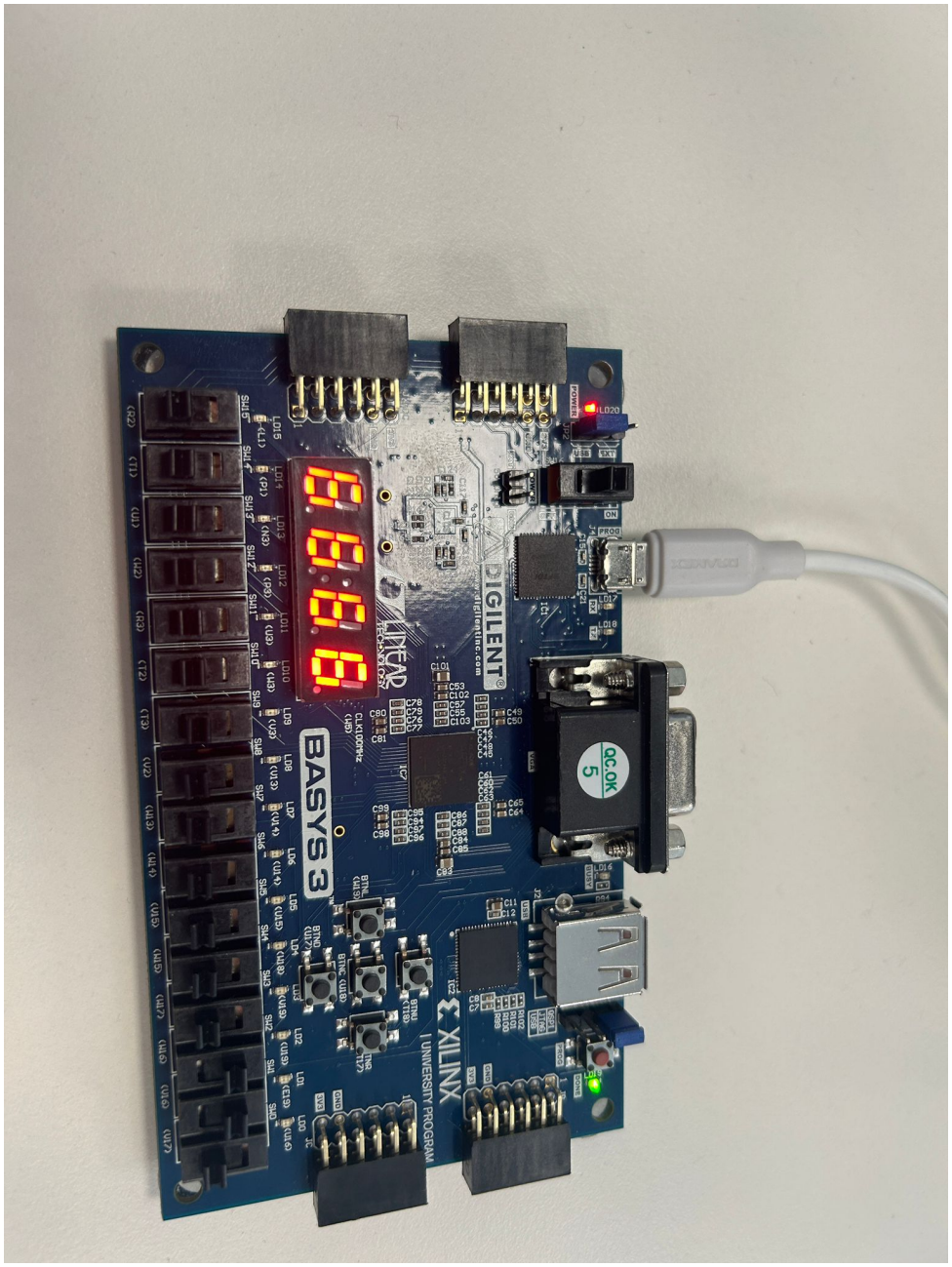

Figure 7: BCD Segment hardware result (six)

4 DISCUSSION

The implementation demonstrated the priority logic inherent in nested `if-else` structures, which was particularly evident in the multiplexer design. A critical observation was the active-low requirement for the seven-segment display segments due to the common anode hardware configuration. Properly mapping the internal HDL signals to the physical FPGA pins using the constraints file was essential for correct hardware operation.

5 CONCLUSION

In this experiment, complex combinational circuits were successfully synthesized and tested on the Basys 3 board. Through the design of the BCD decoder and adder-subtractor, a deeper understanding of Verilog behavioral modeling and FPGA resource utilization was gained. No major difficulties were faced during the signal mapping process.

REFERENCES

[1] Digilent Inc. *Basys 3 FPGA Board Reference Manual*, 2016.

[2] ITU Computer engineering department. Fpga experiment 2. *Digital Logic Journal*, 2026.